

Mathematical Derivation of St lines for Eq – Solver

Abstract :

This work presents mathematical derivations used in the development of an equation solver program. The study focuses on obtaining the general equation of a straight line from different geometric conditions.

Two derivations are discussed: the equation of a line using two points through the point-slope relation, and the equation using the intercept form. In both cases, the coefficients of the general equation, $\mathbf{Ax} \pm \mathbf{By} \pm \mathbf{C} = \mathbf{0}$ are determined and logical conditions are applied to represent the equation correctly for computational implementation.

Concepts Used :

St Line Equations (Coordinate Geometry), Algebraic Expression, Logical Condition (Programming)

Introduction :

Straight line equations are fundamental in coordinate geometry and are widely used in mathematical and computational applications. In many problems, it is necessary to determine the general equation of a line from given geometric information such as two points or axis intercepts

This work presents derivations that convert these geometric inputs into the general form $\mathbf{Ax} \pm \mathbf{By} \pm \mathbf{C} = \mathbf{0}$. These results provide the mathematical basis for implementing straight line equation solving in an equation solver program.

Author Info :

Author : RAI SURAJ (20244098)

Department : Intelligence Computing (Dong-Eui University)

Email : 20244098@g.deu.ac.kr

Combined Equations of a Straight lines (Double - Intercept, Point Slope)

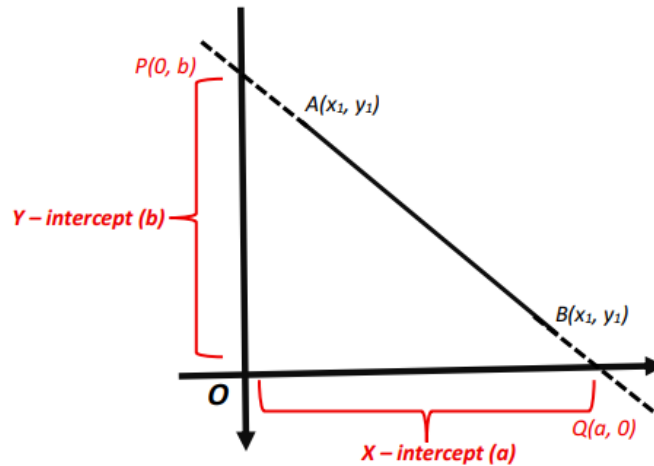


Figure 1: St lines with both intercepts and points.

Equations of a st lines,

$$y - y_1 = m(x - x_1)$$

$$\text{or, } y - y_1 = \frac{\Delta y}{\Delta x} (x - x_1)$$

$$\text{or, } \Delta x(y - y_1) = \Delta y(x - x_1)$$

$$\text{or, } \Delta x \cdot y - \Delta x \cdot y_1 = \Delta y \cdot x - \Delta y \cdot x_1$$

$$\therefore \Delta y \cdot x - \Delta x \cdot y + (\Delta x \cdot y_1 - \Delta y \cdot x_1) = 0$$

Comparing it with $Ax \pm By \pm C = 0$,

$$A = \Delta y, B = \Delta x, C = \Delta x \cdot y_1 - \Delta y \cdot x_1 \quad \dots\dots(i)$$

Then,

Comparing A and B,

if $(A > 0, B > 0)$ else-if $(A > 0, B < 0)$ else-if $(A < 0, B > 0)$ else $(A, B < 0)$

$$Ax - By = 0$$

$$Ax - (-B)y = 0$$

$$(-A)x - By = 0$$

$$(-A)x - (-B)y = 0$$

$$\therefore Ax - By = 0$$

$$Ax + By = 0$$

$$Ax + By = 0$$

$$Ax - By = 0$$

Summarizing all the above conditions,

When $(A > 0, B > 0) \parallel (A < 0, B < 0)$, $Ax - By = 0 \quad \dots\dots eq(ii)$

When $(A > 0, B < 0) \parallel (A < 0, B > 0)$, $Ax + By = 0 \quad \dots\dots eq(iii)$

Again,

Comparing A and C,

if $(A > 0, C > 0)$ else-if $(A > 0, C < 0)$ else-if $(A < 0, C > 0)$ else $(A, C < 0)$

$$Ax + C = 0$$

$$Ax - (-C)y = 0$$

$$(-A)x + C = 0$$

$$(A)x - (-C)y = 0$$

$$\therefore Ax + C = 0$$

$$Ax - C = 0$$

$$Ax - C = 0$$

$$Ax + C = 0$$

Summarizing all the above conditions,

When $(A > 0, C > 0) \parallel (A < 0, C < 0)$, $Ax + C = 0 \quad \dots\dots eq(iv)$

When $(A > 0, C < 0) \parallel (A < 0, C > 0)$, $Ax - C = 0 \quad \dots\dots eq(v)$

Summarizing the previous conditions into Logical Syntax :

For expression of $Ax - By \pm C = 0$

```
if((A>0 && B>0)|| (A<0 && B<0))
    Ax - By  $\pm$  C = 0
else if((A>0 && C>0)|| (A<0 && C>0))
    Ax - By + C = 0
else if((A>0 && C<0) || (A<0 && C>0))
    Ax - By - C = 0
else
    Ax - By = 0
```

For expression of $Ax + By \pm C = 0$

```
if((A>0 && B<0)|| (A<0 && B>0))
    Ax + By  $\pm$  C = 0
else if((A>0 && C<0)|| (A<0 && C>0))
    Ax + By + C = 0
else if((A>0 && C<0)|| (A<0 && C>0))
    Ax + By - C = 0
else
    Ax + By = 0
```

Formulating the above conditions into Logical C Syntax,

```
int main( ){

/* Ask the users for inputs in two options either Point Slope or Double - intercept forms,
using Switch - Case structure. Then, user input points or axis - intercepts are turned
into coefficient value of A and B. */

if(A == 0 && B == 0){
    printf("\nEquation does not exist."); // Both same points, no unique straight line exists.
    return; }

if(A == 0){
    printf("\nY = %d\n",y1); // Horizontal straight line : y = constant.
    return; }

if(B == 0){
    printf("\nX = %d\n",x1); // Vertical straight line : x = constant.
    return; }

// Computes the Highest Common Factor (HCF) to reduce the coefficients A and B.

int hcf = HCF(abs(A),abs(B));
A /= hcf; B /= hcf; // Simplifies the coefficients of the equation.
int C = B*y1 - A*x1; // Calculates the constant term C

// General inclined straight line (A ,B 0)

printf("\nEquation passing through the points (%d,%d) and (%d,%d) is : \n", x1,y1,x2,y2);
// Determines the coefficient of X.

if(abs(A)==1) { strcat(eq,"X "); }
else { sprintf(eq + strlen(eq), "%dX ", abs(A)); }

// Determines the sign and coefficient of Y.

if((A>0 && B<0) || (A<0 && B>0)){
    if(abs(B)==1) { strcat(eq,"+ Y "); }
    else { sprintf(eq + strlen(eq), "+ %dY ", abs(B)); }
}
else{
    if(abs(B)==1) { strcat(eq,"- Y "); }
    else { sprintf(eq + strlen(eq), "- %dY ", abs(B)); }
}

// Determines the sign and magnitude of C.

if(C > 0) { sprintf(eq + strlen(eq), "+ %d = 0", C); }
else if(C < 0) { sprintf(eq + strlen(eq), "- %d = 0", abs(C)); }
else { strcat(eq,"= 0"); }

printf("%s\n", eq); // Displays the final simplified equation.
return 0;
}
```